CS-320 Fall 2026 Section A/C Project 2

In this project, you will create a simulator for a pipelined processor that processes the simplified version of LEGv8 ISA, FOOTv8, that we used in Project 1. You will be provided information about the initial memory contents as input, and your program will need to simulate the program in memory, and write a report about the results of the simulation.

Project Rules

1. All students must work alone. Cooperation is not allowed.

2. Sharing of code between students is considered cheating and will result in a zero grade and an Academic Honesty violation. Source Code will be checked with tools designed to uncover code which is similar even after minor modifications in things like variable names, comments, white space, etc.

3. All project submissions must be written in C code. C++ is not allowed.

4. Your code will be graded on remote.cs.binghamton.edu machines. Please test on either one of these remote machines or the machines in EB-G7, EB-Q22, or EB-N01, which all have the same software configuration.

The FOOTv8 ISA

The FOOTv8 ISA for project 2 is the same as the FOOTv8 ISA for project 1, as follows.

- The instructions supported in FOOTv8 are described in table 1 on page page 2.

- There are 32 32-bit integer registers, named X0-X31.

- All arithmetic operations (ADD, SUB, MUL, and DIV) are integer operations that take two 32 bit two's complement integer operands and produce a single 32 bit two's complement integer result. Division is integer division that rounds towards zero, like C integer division.

Assembly	**Semantics**	**Binary Format**			
		Op	**B1**	**B2**	**B3**
NOP	No-operation	0x00	0x00	0x00	0x00
ADD Rd,Rn,Rm	$R[Rd] = R[Rn] + R[Rm]$	0x02	Rd	Rn	Rm
ADDI Rd,Rn,#Imm	$R[Rd] = R[Rn] + Imm$	0x03	Rd	Rn	Imm
SUB Rd,Rn,Rm	$R[Rd] = R[Rn] - R[Rm]$	0x04	Rd	Rn	Rm
SUBI Rd,Rn,#Imm	$R[Rd] = R[Rn] - Imm$	0x05	Rd	Rn	Imm
MUL Rd,Rn,Rm	$R[Rd] = R[Rn] \times R[Rm]$	0x06	Rd	Rn	Rm
MULI Rd,Rn,#Imm	$R[Rd] = R[Rn] \times Imm$	0x07	Rd	Rn	Imm
DIV Rd,Rn,Rm	$R[Rd] = R[Rn]/R[Rm]$	0x08	Rd	Rn	Rm
DIVI Rd,Rn,#Imm	$R[Rd] = R[Rn]/Imm$	0x09	Rd	Rn	Imm
LD Rd,[Rb,#Imm]	$R[Rd] = Mem[R[Rb] + Imm]$	0x0B	Rd	Rb	Imm
ST Rt,[Rb,#Imm]	$Mem[R[Rb] + Imm] = R[Rt]$	0x0D	Rt	Rb	Imm
EXIT	End of Program	0x0E	0x00	0x00	0x00
MOV Rd,#Imm	$R[Rd] = Imm$	0x11	Rd	Imm	
B #Imm	$PC = PC + (4 \times Imm)$	0x13	0x00	Imm	
CBZ Rt,#Imm	If $Rt == 0$ $PC = PC + (4 \times Imm)$	0x15	Rt	Imm	
CBNZ Rt,#Imm	If $Rt! = 0$ $PC = PC + (4 \times Imm)$	0x17	Rt	Imm	
CBNEG Rt,#Imm	If $Rt < 0$ $PC = PC + (4 \times Imm)$	0x19	Rt	Imm	

Table 1: FOOTv8 Instructions

- The man-readable assembly format has its own features which are not relevant to this project.

- The following are features of the binary machine code format

  - All instructions are four bytes wide. The first byte contains the binary op-code. The remaining three bytes contain the operands of the instruction.

  - The format of the operands depends on the first hex digit of the op-code. If the first digit is 0x0, then the operands are in three one-byte fields. If the first digit is 0x1, there is one one-byte field followed by a two-byte immediate value.

  - Interpretation of the fields is defined in table 1. Register references consist of the numeric register index. For example, 0x0A refers to register X10.

  - The last bit of the binary op-code indicates whether there is an immediate value in the instruction or not. Odd op-codes have immediate values; even op-codes do not. The immediate value is always the last field.

  - All immediate values (Imm in the table) are signed (two's complement) numbers, either 8 bits or 16 bits wide, depending on the instruction.

Program Specifications

Create C code for a single binary executable file named `sim`.

The sim program should take a single run-time parameter that specifies the path and file name of a file which contain the initial memory values.

The sim program will need to simulate execution of FOOTv8 instructions. The first instruction will be in memory at address 0. Simulate starting from the first instruction until an EXIT instruction has been simulated through all pipeline stages.

The sim program should simulate a pipelined FOOTv8 processor, as described in fig. 1. The stages in this illustration operate as follows:

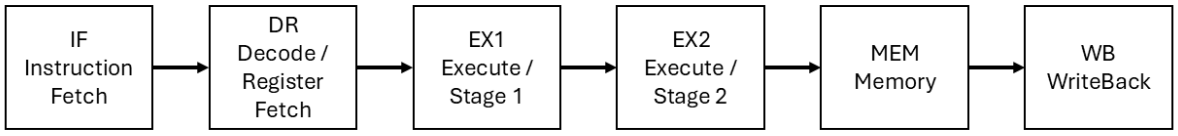


Figure 1: FOOTv8 Processor Pipeline

IF Instruction Fetch. Fetch the instruction at the PC, and update the PC to PC+4, the next sequential instruction.

DR Decode the instruction, and fetch register values for operands.

EX1 First stage of execution. This processes add and subtract instructions, as well as the first half of multiply and divide instructions. It also computes the target address for branches, and the effective address for loads and stores.

EX2 Second stage of execution. This is only needed for multiply and divide instructions, to complete the multiply and divide processing; and for conditional branch instructions, to determine whether the branch is taken or not taken. If the branch is taken, this stage updates the PC.

MEM Memory stage used to read from the data memory for load instructions, and write to data memory for store instructions.

WB Write-back. This stages updates destination registers for arithmetic and load instructions.

Simulate a combined instruction and data memory that contains 2048 four byte words at addresses 0x00000000 to 0x00002000. Initialize the memory using the initial memory values file. Each line of the ASCII initial memory values file contains two 16 digit hexadecimal numbers. The first number contains a memory address divisible by 4. The second number contains the four byte initial value starting at that address. Any word in memory which does not appear in the initial memory file should be initialized to zero. There is no need to check the validity of the memory values file. Only valid memory values files will be used to test your simulator.

The architectural registers, X0-X31, should be initialized to zero.

After initialization, your processor should fetch the first instruction in cycle 0, and continue processing instructions through the pipeline until the cycle in which an EXIT instruction reaches the final, write-back stage.

Project 2 does not support data forwarding. A data hazard is detected in the DR stage. When a data hazard is detected, the DR and IF stages halt until the data hazard is resolved. A data hazard is resolved when the pending register update gets to the WB stage. The register write occurs early in the cycle, so a register read in the same cycle gets the new value.

Project 2 also does not support any branch prediction. A control hazard is detected in the EX2 stage and only when a branch is taken. At that point, instructions in the IF, DR, and EX1 stages must be squashed, the PC is updated with the correct branch target, and processing resumes.

Your simulation needs to keep track of all register values throughout the simulation, including the inter-stage registers required to run the pipeline. You should also keep track of the number of cycles, the number of instructions that are simulated completely (not counting squashed instructions) for each instruction op-code (other than NOP) as well as the total number of instructions (other than NOP).

The sim program may print messages to standard output, but must end standard output with a standard set of reports. These reports must be formatted correctly since your grade will be evaluated by diff'ing your output with the correct output. The reporting lines should start with:

```
=== Simulation Reports ===
```

See the log files in `proj2/correct_logs` in the files provided to you for examples of the expected reports.

Working the Project

The proj2.tar.gz file is available for download from Brightspace. Download and untar this file to create an initial **proj2** directory. The files in this directory are designed to work in the remote.cs.binghamton.edu environment. They may work in other UNIX environments as well, but make sure you test on the remote machines.

Design and code your simulator by writing code in the code sub-directory, and updating the code/Makefile to build the executable file, `../sim`. You may add new code files in the code sub-directory, but if you do so, update the Makefile in the code directory so that your code builds correctly.

Test and debug your code from the proj2 directory by running `make test` to run a simple test, `make gdb` to run a simple test under gdb, `make checkMem` to run a simple test under valgrind to check memory usage. You may want to modify the Makefile to modify the initial memory input file used in the simple test. Run `make fullTest` to run all initial memory input files provided to you.

When you are ready to submit, run `make submit` in the proj2 directory. This will create a `proj2_code.tar.gz` file in the proj2 directory that contains the contents of your code sub-directory. Upload this tar file in the submission area on Brightspace.

Warning: Brightspace prohibits the presence of executable files when you upload. If you get errors uploading your tar file from Brightspace, you may have left an executable file in the code sub-directory.

You may submit as many times as you want before the project deadline. Only the last submission will be graded.

Grading Criteria

Your proj2_code.tar.gz file will be downloaded and unzipped on a remote.cs server. We will then build your sim executable. If the compiler produces error messages while building your code, we will attempt to fix the problem. After your code is compiles, it will be run on the remote.cs servers using five different initial memory files, two of which has been provided to you; the other three are FOOTv8 initial memory files you have not seen.

By default, you start with 100 points for the project. The following deductions from the maximum grade are possible:

-15 For each fix required to get an error-free compile, up to a maximum of four fixes.

-10 If there are any warning messages from the compiler for your code, running with the `-Wall` compiler option. Warning messages often indicate potential bugs; it's worth fixing them!

-10 If there are output formatting errors in the simulation reports.

-3 For each incorrect result in the simulation reports produced by your program, up to a maximum of -15 per test case.

-100 If your code matches or is very similar to another student's code, or code available on the internet. This is an academic honesty violation.

-100 If you make no attempt to solve the problem.